

GET STARTED

- Welcome to SP-API Documentation
- What is the Selling Partner API?
- Selling Partner API Onboarding Overview >

- Terminology
- Frequently Asked Questions >

CHANGELOG

- SP-API Release Notes
- SP-API Deprecation Schedule
- SP-API Product Metadata Updates

DIRECTORIES

- SP-API Models
- SP-API Seller Use Cases

Create and Filter Notification Subscriptions

Use the Notifications API to create and filter subscriptions with CEL-based filter expressions.

Learn how to use the Notifications API to create subscriptions with filter expressions that control which event notifications you receive. The `filterExpression` field accepts a [Common Expression Language \(CEL\)](#) expression that evaluates against each event's payload and returns only the events that evaluate to `true`.

When an event is generated, the filter expression is evaluated against the event payload, and only events that satisfy the filter criteria are forwarded to your endpoint.

Prerequisites

To complete this tutorial, you need:

- Authorization from the selling partners for whom you are making calls. Refer to [Authorizing Selling Partner API applications](#) for more information.

Eligible notification types

The following notification types support `filterExpression`:

Notification type	<code>filterExpression</code> supported
<code>ACCOUNT_STATUS_CHANGED</code>	Yes
<code>ANY_OFFER_CHANGED</code>	Yes
<code>B2B_ANY_OFFER_CHANGED</code>	Yes
<code>DETAIL_PAGE_TRAFFIC_EVENT</code>	Yes
<code>EXTERNAL_FULFILLMENT_SHIPMENT_STATUS_CHANGE</code>	Yes
<code>FBA_INVENTORY_AVAILABILITY_CHANGES</code>	Yes
<code>FBA_OUTBOUND_SHIPMENT_STATUS</code>	Yes
<code>FEED_PROCESSING_FINISHED</code>	Yes
<code>FEE_PROMOTION</code>	Yes
<code>FULFILLMENT_ORDER_STATUS</code>	Yes
<code>ITEM_INVENTORY_EVENT_CHANGE</code>	Yes
<code>ITEM_SALES_EVENT_CHANGE</code>	Yes
<code>ORDER_CHANGE</code>	Yes
<code>PRICING_HEALTH</code>	Yes
<code>REPORT_PROCESSING_FINISHED</code>	Yes

SP-API Vendor Use Cases	
Seller Central URLs	
Vendor Central URLs	
REGISTRATION	
SP-API Registration Overview	>
Developer Registration Request Status	
Third-Party Provider Registration	>
Service Provider Registration and Role Approval Process Overview	
Website Guidelines for Public Developers and Service Providers	
User Permissions for Service Providers	
Register your Application	
Update your Application Information	
Rotate your Application's LWA Credentials	>
View your Application Information and Credentials	
Learn How Sellers Authorize Service Providers	
Selling Partner API Roles	>
Policies and Agreements	
About Facial Data	
AUTHORIZATION	
Authorize Applications	>
Renew Authorizations	
Revoke Authorizations	
Authorization Limits	
Special Authorization Types	>
INTEGRATION	
Building Robust Amazon SP-API Applications	
SP-API SDKs	>

Notification type	filterExpression supported
TRANSACTION_UPDATE	Yes
LISTINGS_ITEM_ISSUES_CHANGE	No
LISTINGS_ITEM_MFN_QUANTITY_CHANGE	No
LISTINGS_ITEM_STATUS_CHANGE	No
PRODUCT_TYPE_DEFINITIONS_CHANGE	No
BRANDED_ITEM_CONTENT_CHANGE	No
ITEM_PRODUCT_TYPE_CHANGE	No

 Note

You cannot use `filterExpression` and `eventFilter` together. Use one or the other. Use filter expressions to replace existing `eventFilter` conditions under `processingDirective`. If you need both aggregation and filtering, continue to use `eventFilter`.

Supported CEL operators

The following table lists the CEL operators that you can use in a `filterExpression`:

Operator type	Supported	Not supported
Logical	<code>!</code> (NOT), <code>&&</code> (AND), <code> </code> (OR), <code>? :</code> (ternary)	N/A
Comparison	<code>==</code> , <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>in</code>	N/A
Comprehension macros	<code>has(e.f)</code> , <code>e.all(x, p)</code> , <code>e.exists(x, p)</code> , <code>e.exists_one(x, p)</code>	<code>e.filter(x, p)</code> , <code>e.map(x, t)</code>
String functions	<code>contains()</code> , <code>startsWith()</code> , <code>endsWith()</code> , <code>matches()</code> , <code>size()</code>	N/A
List/Map operations	<code>in</code> , <code>size()</code>	<code>[]</code> (indexing)
Arithmetic	N/A	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>

Step 1. Identify the notification payload fields

Before you create a filter expression, review the payload structure of the notification type you want to filter.

 Note

Filter expressions can only reference fields within the notification payload. The filter returns only events that evaluate to `true`. You can write filter expressions on optional fields. However, you must check whether the field is present before you use it, or the filter fails at runtime. Use the `has()` macro for field presence checks.

Step 2. Write the CEL filter expression

Write a CEL expression that targets the payload fields you identified in Step 1.

Filtering on optional fields with `has()`

Some notification payloads contain optional fields that are only present under certain conditions. If your `filterExpression` references a field that is absent from the payload, the

filter evaluation fails at runtime and the event is dropped. To safely filter on optional fields, use the `has()` macro to check for field presence before accessing the field.

 Note

If `has()` is not used as in the example below, the request will fail when `FulfillmentShipment` is not present:

```
FulfillmentOrderStatusNotification.FulfillmentShipment.FulfillmentShipmentStatus == "Shipped"
```

The `FulfillmentShipment` field in the `FULFILLMENT_ORDER_STATUS` notification is optional. It is only present when the notification contains shipment information. If you want to filter on a field inside `FulfillmentShipment`, you must first check that it exists.

```
has(FulfillmentOrderStatusNotification.FulfillmentShipment) &&
FulfillmentOrderStatusNotification.FulfillmentShipment.FulfillmentShipmentStatus == "Shipped"
```

Step 3. Create the subscription with a filter expression

Call the `createSubscription` operation and include the `filterExpression` field in the `processingDirective` object.

The following examples show sample notification payloads and the corresponding `createSubscription` requests with `filterExpression`.

FULFILLMENT_ORDER_STATUS

```
JSON
{
  "NotificationVersion": "1.0",
  "NotificationType": "FULFILLMENT_ORDER_STATUS",
  "PayloadVersion": "1.0",
  "EventTime": "2020-01-11T00:09:53.109Z",
  "Payload": {
    "FulfillmentOrderStatusNotification": {
      "SellerId": "A3TH9S8BH6GOGM",
      "EventType": "Shipment",
      "StatusUpdatedDateTime": "2020-01-11T00:09:53.109Z",
      "SellerFulfillmentOrderId": "ext-order-id-12345",
      "FulfillmentOrderStatus": "Complete",
      "FulfillmentShipment": {
        "FulfillmentShipmentStatus": "Shipped",
        "AmazonShipmentId": "DZRSmwG2N",
        "EstimatedArrivalDateTime": "2020-01-14T22:59:59Z",
        "FulfillmentShipmentPackages": [{
          "PackageNumber": 1,
          "CarrierCode": "UPS",
          "TrackingNumber": "1Z999AA10123456784"
        }]
      }
    }
  },
  "NotificationMetadata": {
    "ApplicationId": "amzn1.sellerapps.app.f1234566-aaec-55a6-b123-bcb752069ec5",
    "SubscriptionId": "7d78cc50-95c8-4641-add7-10af4b1fedc9",
    "PublishTime": "2020-01-11T00:02:50.501Z",
    "NotificationId": "2012e8e5-b365-4cb1-9fd8-be9dfc6d5eaf"
  }
}
```

Delete an Application From Your Developer Account

CALLING THE SP-API

Configuration Details

Call Structure

Development Tools

Performance Management

SOLUTION PROVIDER PORTAL

Manage Users in Solution Provider Portal

Manage Your Business And Contact information in Solution Provider Portal

Update Your Developer Profile in Solution Provider Portal

Unify Your Developer Accounts

Verify Your Identity in Solution Provider Portal

API Usage Metrics in Solution Provider Portal

TUTORIALS

Tutorial: Subscribe to the ORDER_CHANGE Notification

Tutorial: Test Selling Partner API Endpoints

Tutorial: Automate your SP-API Calls Using a C# SDK

Tutorial: Automate your SP-API Calls Using a Java SDK

Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js

Tutorial: Automate your SP-API Calls Using a Python SDK

Tutorial: Automate your SP-API Calls using a prebuilt C# SDK

Tutorial: Automate your SP-API Calls using prebuilt Java SDK

Tutorial: Automate your SP-API Calls using a prebuilt JavaScript SDK

Tutorial: Automate your SP-API Calls using prebuilt PHP SDK

Tutorial: Automate your SP-API calls using a prebuilt Python SDK

Tutorial: Authorize Multiple Vendor Central Accounts with a Single SP-API Application

Tutorial: Retrieve Merchant Shipping Templates

Tutorial: Grant the SP-API Permission to an Amazon SQS Queue

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

TROUBLESHOOTING

Troubleshoot SP-API Errors

URL Encoding

Resolve Common HTTP and Authorization Error Codes

External Fulfillment Errors [↗](#)

Listings Items API Issues Troubleshooting [↗](#)

SELLING PARTNER APPSTORE

What is the Selling Partner Appstore?

List Your App on the Selling Partner Appstore

Edit Your Appstore Listing

Check Listing Status

Appstore Ratings and Reviews

Press Releases and Promotions

SERVICE PROVIDER NETWORK

List Your Service

SECURITY AND COMPLIANCE

SP-API Security and Compliance Overview

Amazon Selling Partner API Guard Implementation Guide [➤](#)

Sample createSubscription request with filterExpression

JSON

```
{
  "payloadVersion": "1.0",
  "destinationId": "sampleId",
  "processingDirective": {
    "filterExpression": "FulfillmentOrderStatusNotification.EventType
== \"ORDER\" && FulfillmentOrderStatusNotification.FulfillmentOrderStatus
== \"COMPLETE\""
  }
}
```

REPORT_PROCESSING_FINISHED

JSON

```
{
  "notificationVersion": "2020-09-04",
  "notificationType": "REPORT_PROCESSING_FINISHED",
  "payloadVersion": "2020-09-04",
  "eventTime": "2020-07-14T03:35:13.214Z",
  "payload": {
    "reportProcessingFinishedNotification": {
      "sellerId": "A3TH9S8BH6G0GM",
      "accountId": "amzn1.merchant.o.A3TH9S8BH6G0GM",
      "reportId": "54517018502",
      "reportType": "GET_FLAT_FILE_ACTIONABLE_ORDER_DATA",
      "processingStatus": "CANCELLED",
      "reportDocumentId": "amzn1.tortuga.3.edbcd0d8-3434-8222-1234-52ad8ade1208.REP4567URI9BMZ"
    },
    "notificationMetadata": {
      "applicationId": "amzn1.sellerapps.app.aaccczf-4455-4b7c-4422-664ecacdd336",
      "subscriptionId": "subscription-id-d0e9e693-c3ad-4373-979f-ed4ec98dd746",
      "publishTime": "2020-07-13T19:45:04.284Z",
      "notificationId": "d0e9e693-c3ad-4373-979f-ed4ec98dd746"
    }
  }
}
```

Sample createSubscription request with filterExpression

JSON

```
{
  "payloadVersion": "1.0",
  "destinationId": "sampleId",
  "processingDirective": {
    "filterExpression":
"reportProcessingFinishedNotification.processingStatus == \"CANCELLED\" &&
reportProcessingFinishedNotification.reportType ==
\"GET_FLAT_FILE_ACTIONABLE_ORDER_DATA\""
  }
}
```

Migrate existing subscriptions to filter expressions

This section explains how to add filtering to your existing subscriptions and how to migrate from `eventFilter` to `filterExpression`.

! Important

VAT Calculation Service >

Amazon Seller Data Access
Best Practices >

A+ CONTENT API
A+ Content API >

A+ Content Examples

AMAZON WAREHOUSING AND DISTRIBUTION API
Amazon Warehousing and Distribution API >

Amazon Warehousing and Distribution API v2024-05-09
Reference

APP INTEGRATIONS API
App Integrations API >

Create and Filter Notification Subscriptions

- You cannot update an existing subscription to add a `filterExpression` . You must delete the existing subscription and create a new one.
- You cannot use `eventFilter` and `filterExpression` together in the same subscription.
- Existing subscriptions without a `filterExpression` continue to receive all events. There is no change to current behavior unless you explicitly create a new subscription with a `filterExpression` .

Migration approach

To avoid missing events during migration, run your new filtered subscription in parallel with your existing subscription before switching over.

1. Create a new subscription with `filterExpression` alongside your existing subscription (use a separate destination or the same destination if your processor can handle both).
2. Receive and process filtered notifications in parallel but do not use them for business logic yet.
3. Confirm that filtered notifications match expectations: correct events are delivered, no unexpected gaps, and your processor handles them correctly. Run for 1–2 weeks.
4. Once validated, delete your old subscription using the `deleteSubscription` operation.

Note

If you cannot run two subscriptions in parallel, delete the old subscription and create the new one. Be aware there may be a brief gap in notifications during the switch.

Scenario 1: Add filtering to an existing subscription (no prior filtering)

Use this approach when you currently receive all events for a notification type and want to filter to only specific events.

Use case: You subscribe to `ANY_OFFER_CHANGED` and receive notifications for all offer changes. You want to receive only notifications triggered by a Featured Offer change.

Sample `ANY_OFFER_CHANGED` notification payload

JSON

```
{
  "NotificationVersion": "1.0",
  "NotificationType": "ANY_OFFER_CHANGED",
  "PayloadVersion": "1.0",
  "EventTime": "2020-07-13T19:42:04.284Z",
  "Payload": {
    "AnyOfferChangedNotification": {
      "SellerId": "A3TH9S8BH6GOGM",
      "OfferChangeTrigger": {
        "MarketplaceId": "ATVPDKIKX0DER",
        "ASIN": "B08N5WRWNW",
        "ItemCondition": "New",
        "TimeOfOfferChange": "2020-07-13T19:42:04.284Z",
        "OfferChangeType": "FeaturedOffer"
      },
      "Summary": {
        "NumberOfOffers": [
          {
            "Condition": "new",
            "FulfillmentChannel": "Merchant",
            "OfferCount": 10
          }
        ]
      }
    }
  }
}
```

- APPLICATION MANAGEMENT API
 - Application Management API
- CATALOG ITEMS API
 - Catalog Items API
 - Catalog Items API Rate Limits
 - Catalog Items API v2022-04-01 Reference
- CUSTOMER FEEDBACK API
 - Customer Feedback API
 - Customer Feedback API Rate Limits
- DATA KIOSK
 - Data Kiosk API
 - Data Kiosk Schema Explorer User Guide
 - Data Kiosk Query Processing Finished Notification
 - Building Data Kiosk workflows guide
 - Vendor Analytics Dataset Use Case Guide
 - Schedule Data Kiosk Queries
- DELIVERY BY AMAZON API
 - Delivery by Amazon API
- EASY SHIP API
 - Easy Ship API
- EXTERNAL FULFILLMENT
 - External Fulfillment Inventory API

```
    "TotalBuyBoxEligibleOffers": 100
  },
  "Offers": [
    {
      "SellerId": "A3TH9S8BH6G0GM",
      "SubCondition": "New",
      "IsFulfilledByAmazon": true,
      "IsFeaturedMerchant": true
    }
  ]
},
"NotificationMetadata": {
  "ApplicationId": "amzn1.sellerapps.app.f1234566-aaec-55a6-b123-bcb752069ec5",
  "SubscriptionId": "7d78cc50-95c8-4641-add7-10af4b1fedc9",
  "PublishTime": "2020-07-13T19:42:04.284Z",
  "NotificationId": "d0e9e693-c3ad-4373-979f-ed4ec98dd746"
}
```

Step 1. Create a new subscription with a filter expression

Call the `createSubscription` operation with a `filterExpression` in the `processingDirective`.

```
JSON
{
  "payloadVersion": "your-payload-version",
  "destinationId": "your-destination-id",
  "processingDirective": {
    "filterExpression":
      "AnyOfferChangedNotification.OfferChangeTrigger.OfferChangeType == \"FeaturedOffer\""
  }
}
```

Step 2. Run in shadow mode and validate

Run your new filtered subscription in parallel with your existing unfiltered subscription. Process the filtered notifications but do not use them for business logic yet. Confirm that the expected events are delivered, no events are unexpectedly missing, and your processor handles the filtered payload correctly.

Step 3. Delete your old subscription

After validation is complete, call the `deleteSubscription` operation to remove your old subscription.

```
DELETE /notifications/v1/subscriptions/ANY_OFFER_CHANGED
```

You now receive `ANY_OFFER_CHANGED` notifications only when the offer change is triggered by a Featured Offer change. All other offer change types (Internal, External) are filtered out.

Scenario 2: Migrate from `eventFilter` to `filterExpression` (`ANY_OFFER_CHANGED`)

Use this approach when you currently use `eventFilter` and want to migrate to `filterExpression` for more flexible filtering.

Use case: You currently use `eventFilter` to filter `ANY_OFFER_CHANGED` notifications to a specific marketplace. You want to migrate to `filterExpression` to also filter by offer change type.

Your current subscription (with `eventFilter`):

```
JSON
```

External Fulfillment Returns API >

External Fulfillment Shipping API >

External Fulfillment Errors

External Fulfillment APIs Rate Limits

FULFILLMENT BY AMAZON (FBA)

FBA Inventory API >

FBA Inventory Dynamic Sandbox Guide

FEEDS API

Feeds API >

Feed Type Values >

```
{
  "payloadVersion": "your-payload-version",
  "destinationId": "your-destination-id",
  "processingDirective": {
    "eventFilter": {
      "eventFilterType": "ANY_OFFER_CHANGED",
      "marketplaceIds": ["ATVPDKIKX0DER"]
    }
  }
}
```

Step 1. Create a new subscription with `filterExpression`Convert your `eventFilter` marketplace filter to a CEL expression:

JSON

```
{
  "payloadVersion": "your-payload-version",
  "destinationId": "your-destination-id",
  "processingDirective": {
    "filterExpression":
      "AnyOfferChangedNotification.OfferChangeTrigger.MarketplaceId ==
      \"ATVPDKIKX0DER\""
  }
}
```

To combine marketplace filtering with offer change type filtering:

JSON

```
{
  "payloadVersion": "your-payload-version",
  "destinationId": "your-destination-id",
  "processingDirective": {
    "filterExpression":
      "AnyOfferChangedNotification.OfferChangeTrigger.MarketplaceId ==
      \"ATVPDKIKX0DER\" &&
      AnyOfferChangedNotification.OfferChangeTrigger.OfferChangeType ==
      \"FeaturedOffer\""
  }
}
```

Feeds JSON Schemas >

Feeds API Best Practices

Feeds API FAQ

Feeds API Rate Limits

FINANCES

Finances API >

Step 2. Run in shadow mode and validate

Run your new filtered subscription in parallel with your existing `eventFilter` subscription. Process the filtered notifications but do not use them for business logic yet. Confirm that the expected events are delivered, no events are unexpectedly missing, and your processor handles the filtered payload correctly.

Step 3. Delete your old subscription

After validation is complete, delete your old subscription:

Transfers API >

DELETE /notifications/v1/subscriptions/ANY_OFFER_CHANGED

FULFILLMENT INBOUND API

Fulfillment Inbound API >

You now receive `ANY_OFFER_CHANGED` notifications only for the US marketplace (ATVPDKIKX0DER) where the change was triggered by a Featured Offer update.

Note

If you currently rely on `aggregationSettings` in your `eventFilter` (for example, to limit notification frequency), you cannot replicate this behavior with `filterExpression`. Aggregation is only available with `eventFilter`. If you need both aggregation and filtering, continue to use `eventFilter`.

Fulfillment Inbound API FAQ

 Updated 19 days ago

Migrating Fulfillment Inbound workflows

Fulfillment Inbound API Rate Limits

← Notifications API

Set up notifications using the Amazon EventBridge workflow →

FULFILLMENT OUTBOUND API

Fulfillment Outbound Dynamic Sandbox Guide

Fulfillment Outbound API >

Did this page help you?  Yes  No

[Policies and Agreements](#)

[Privacy notice](#) | [Conditions of use](#)

Fulfillment Outbound API Rate Limits

MCF Best Practices

INVOICING

Invoices API >

Invoices API FAQ

LISTINGS

Listings Items API >

Listings Items API Rate Limits

Listings Restrictions API >

Listings Restrictions API Rate Limits

Manage Product Listings with the Selling Partner API

Building Listings Management Workflows Guide

Understanding Amazon listing status and seller-fulfilled inventory management

Listings Management Workflow Migration >

Manage Amazon Haul, Advanced Multiple-Offer, and Multiple-Fulfillment Use Cases

Listings APIs FAQ

Product Type Definitions API >

Product Type Definitions API Rate Limits

MERCHANT FULFILLMENT API

Merchant Fulfillment API >

MESSAGING API

Messaging API >

NOTIFICATIONS API

Notifications API >

Create and Filter Notification Subscriptions

Set up notifications using the Amazon EventBridge workflow

Set up notifications using the Amazon Simple Queue Service workflow

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification ↗

ORDERS API

Orders API >

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier ↗

Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API >

PRODUCT PRICING API

Product Pricing API >

Product Pricing API and Notifications FAQ

Price Adjustment Automation Workflows Guide

Manage automated pricing rules with SP-API

REPLENISHMENT API

Replenishment API >

REPORTS API

Reports API >

Report Type Values >

Reports JSON Schemas ↗

Reports API FAQ

SALES API

Sales API >

SELLER WALLET API

Seller Wallet API >

Seller Wallet API Rate Limits

SELLERS API

Sellers API >

SERVICES API

Services API >

SHIPMENT INVOICING API

Shipment Invoicing API >

SHIPPING API

Shipping API v2 Resources ↗

Shipping API v1 Reference

SOLICITATIONS API

Solicitations API >

SUPPLY SOURCES API

Supply Sources API >

Multi-Location Inventory Integration Guide

Supply Sources API Rate Limits

TOKENS API

Tokens API >

UPLOADS API

Uploads API >

VEHICLES API

Vehicles API >

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API >

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API >

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API >

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API >

VENDOR DIRECT FULFILLMENT PAYMENTS API

Vendor Direct Fulfillment Payments API >

VENDOR RETAIL PROCUREMENT INVOICES API

Vendor Retail Procurement Invoices API >

VENDOR RETAIL PROCUREMENT ORDERS API

Vendor Retail Procurement Orders API >

VENDOR RETAIL PROCUREMENT SHIPMENTS API

Vendor Retail Procurement Shipments API >

**VENDOR RETAIL PROCUREMENT TRANSACTION STATUS
API**

Vendor Retail Procurement Transaction Status API >

